

Frog Feast:

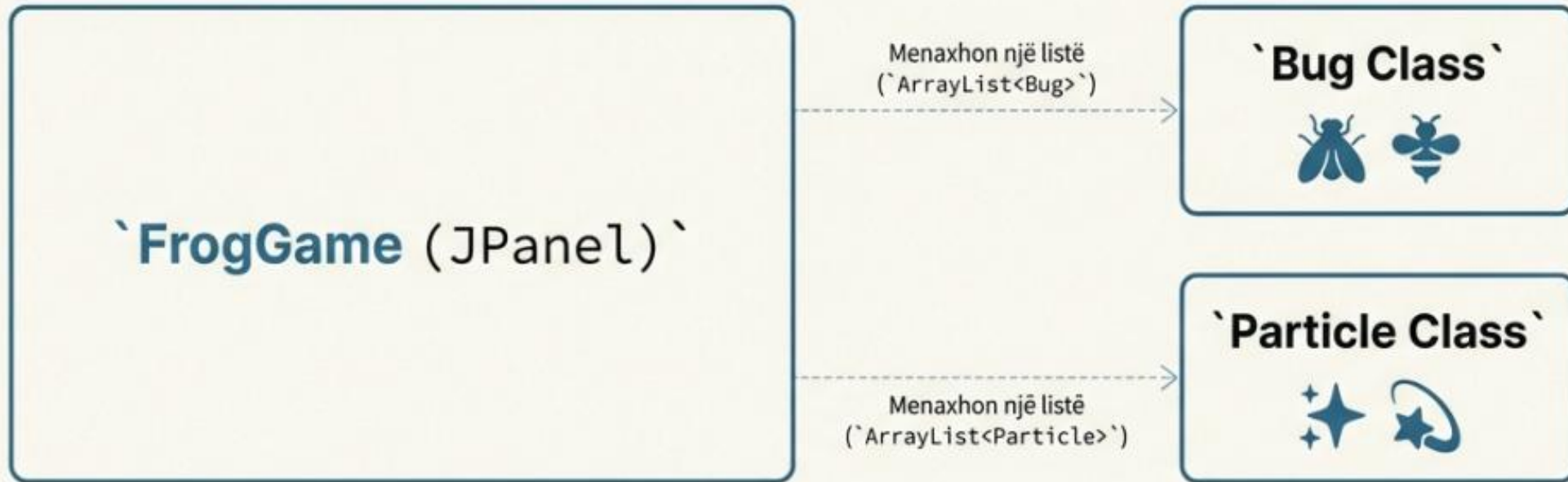


Mirë se vini në analizën teknike të 'Frog Feast', një lojë e thjeshtë 2D e ndërtuar në Java.

- 🎯 Kap **3** miza (🦋) për të fituar.
- 🐝 Shmang bletët.
- ❌ Humb pas **5** gjuajtjeve të pasakta.

Në këtë prezantim, ne do të zbërthejmë kodin burimor për të kuptuar se si çdo element, nga lëvizja e insekteve te mekanika e gjuhës, funksionon së bashku.

Anatomia e një Loje të Thjeshtë



`FrogGame`

Klasa kryesore që shërben si skena dhe motori i lojës. Ajo menaxhon gjendjen e lojës, përpunon inputin e përdoruesit dhe koordinon të gjitha elementet e tjera.



`Bug`

Një klasë model që përfaqëson insektet (mizat dhe bletët). Secili objekt **`Bug`** ka logjikën e tij për lëvizjen dhe vizatimin.



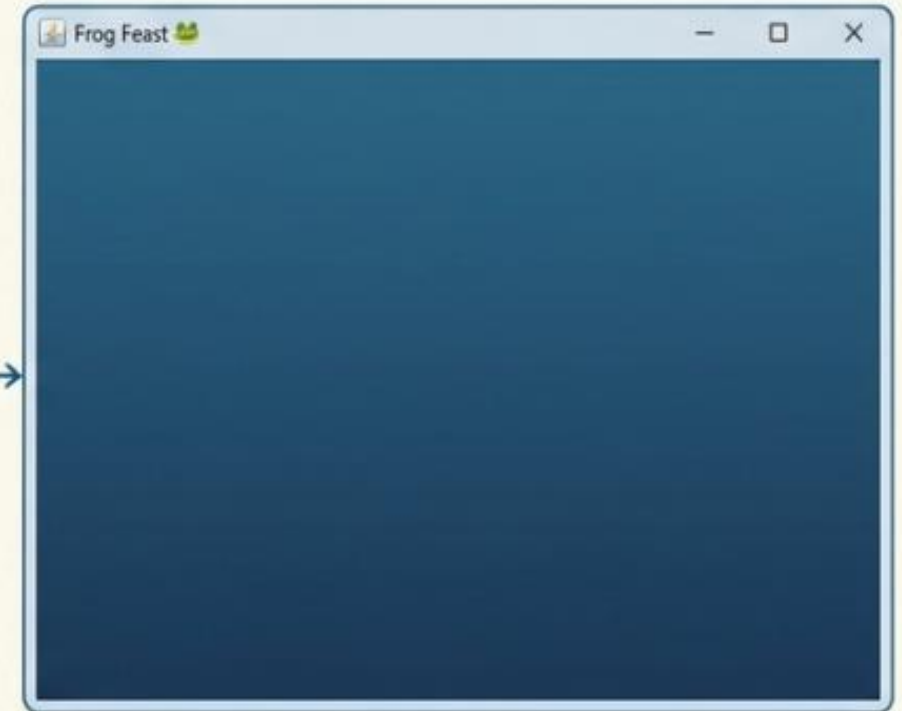
`Particle`

Një klasë e thjeshtë për efektet vizuale. Përdoret për të krijuar shpërthimin e grimcave kur një mizë kapet.

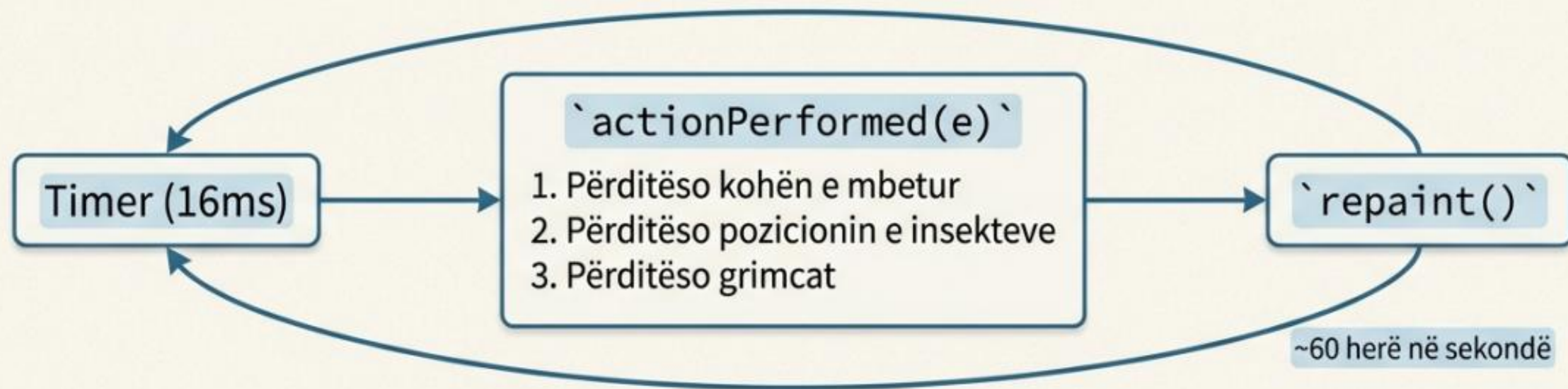
Ndërtimi i Pellgut Dixhital

```
public static void main(String[] args) {  
    JFrame frame = new JFrame("Frog Feast 🐸");  
    FrogGame game = new FrogGame();  
    frame.add(game);  
    frame.pack();  
    // ...  
    frame.setVisible(true);  
}
```

```
// Në konstruktor  
setPreferredSize(new Dimension(GAME_WIDTH, GAME_HEIGHT));  
setBackground(new Color(42, 95, 127));  
  
// Në paintComponent  
GradientPaint gradient = new GradientPaint(0, 0, new Color(42, 95, 127),  
    0, GAME_HEIGHT, new Color(26, 58, 95));  
g2d.setPaint(gradient);  
g2d.fillRect(0, 0, GAME_WIDTH, GAME_HEIGHT);
```



Rrahja e Zemrës së Lojës: Cikli Kryesor



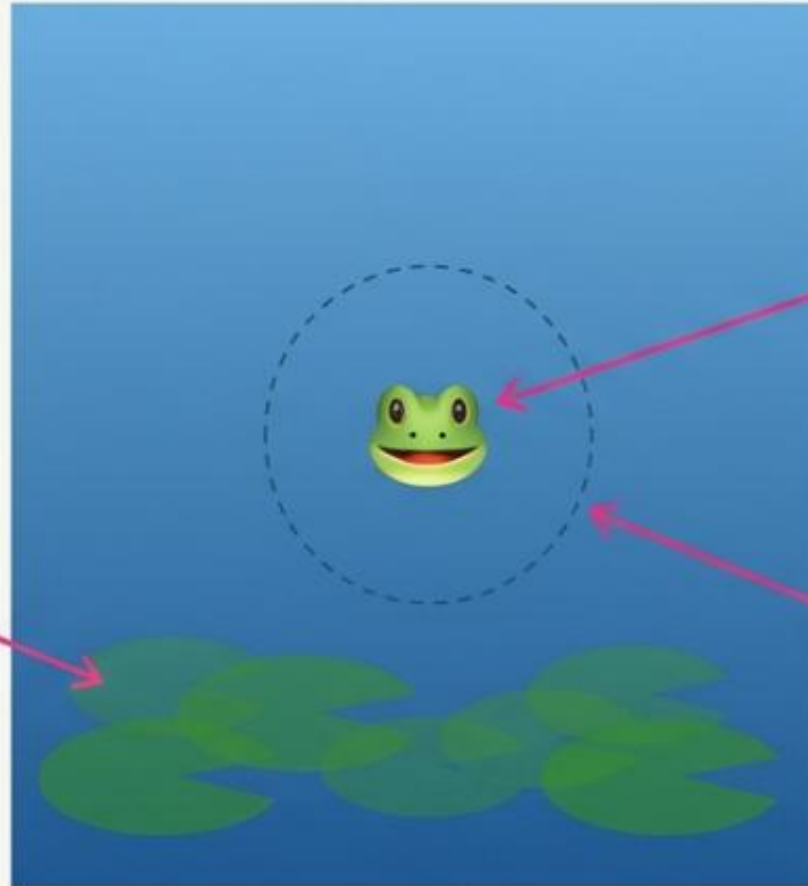
```
// Në konstruktor
this.gameTimer = new Timer(16, this); // ~60 FPS
gameTimer.start();

// Metoda kryesore e ciklit
@Override
public void actionPerformed(ActionEvent e) {
    if (gameActive) {
        // ... logjika e përditësimit ...
    }
    repaint(); // Rindërto ekranin
}
```

Motori i lojës është një `Timer` që ekzekutohet çdo 16 milisekonda. Sa herë që 'rreh', ai thërret metodën `actionPerformed`, e cila përditëson gjendjen e të gjitha elementeve të lojës dhe më pas kërkon një rindërtim të ekranit.

Pikturimi i Skenës me Kod

```
// Vizato zambakët e ujit  
g2d.setColor(new Color(34, 139, 34, 25));  
g2d.fillOval(x - 40, y - 30, 80, 60);
```



```
// Vizato bretkosën  
g2d.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 60));  
g2d.drawString("🐸", frogX - 36, frogY + 20);
```

```
// Vizato rrezen e kapjes  
g2d.drawOval(frogX - TONGUE_REACH, frogY - TONGUE_REACH,  
             TONGUE_REACH * 2, TONGUE_REACH * 2);
```

Metoda `paintComponent` është kanavaca jonë. Ajo ekzekutohet sa herë që thirret `repaint()` dhe vizaton çdo element në mënyrë sekuenciale, nga sfondi te bretkosa, duke përdorur objektin `Graphics2D`.

Aktorët në Skenë: Miza dhe Bleta



```
class Bug {  
    String type; // "fly" ose "bee" ←  
    double x, y; // Pozicioni  
    double vx, vy; // Shpejtësia (vektori)  
  
    Bug(String type) {  
        this.type = type;  
        reset(); // Cakto pozicionin dhe shpejtësinë fillestare  
    }  
    // ... metodat update(), draw(), isHit() ...  
}
```



```
void draw(Graphics2D g) {  
    g.setFont(new Font("Segoe UI Emoji", Font.PLAIN, type.equals("fly") ? 24 : 28));  
    String emoji = type.equals("fly") ? "🪰" : "🐝";  
    g.drawString(emoji, (int) x - 12, (int) y + 8);  
}
```

Çdo insekt në lojë është një instancë e klasës `Bug`. Atributi `type` përcakton sjelljen dhe pamjen e tij. Kjo qasje e orientuar nga objektet na lejon të menaxhojmë me lehtësi entitete të shumta në lojë.

Sjellja e Insekteve: Gjenerimi dhe Lëvizja

Gjenerimi (Spawning)



```
// Gjenero miza periodikisht
Timer flySpawner = new Timer(5000, e -> {
    if (gameActive && countFlies() < 7) {
        bugs.add(new Bug("fly"));
    }
});

// Gjenero bleta periodikisht
Timer beeSpawner = new Timer(10000, e -> {
    if (gameActive && random.nextDouble() < 0.3) { // 30% shans
        bugs.add(new Bug("bee"));
    }
});
```

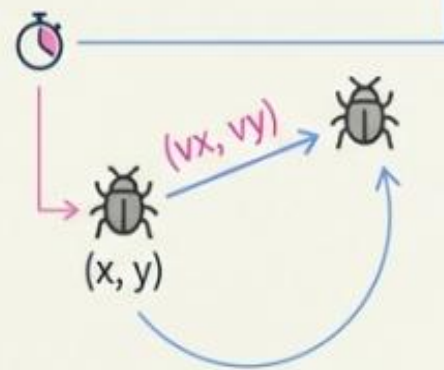


Insektet nuk janë të paracaktuara. `Timer`-a të veçantë shtojnë miza dhe bleta në lojë në intervale të rregullta, duke krijuar një  një përvojë dinamike.

Lëvizja

```
void update() {
    x += vx;
    y += vy;
    // Logjika për t'u kthyer në ekran kur dalin jashtë
    if (x < -50) x = GAME_WIDTH + 50;
    // ...
}
```

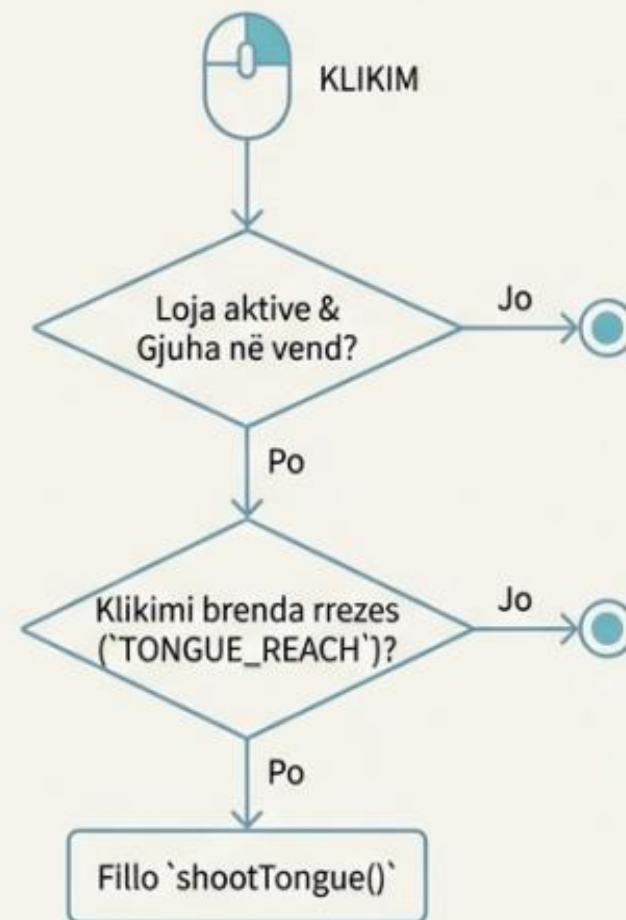
Në çdo 'rrahje' të ciklit të lojës, thirret metoda `update` për çdo insekt. Kjo lëvizje e thjeshtë vektoriale (`pozicioni` `pozicioni += shpejtësia`) krijon iluzionin e fluturimit.



Ura Lidhëse: Si Lojtari Ndërvepron me Lojën

```
@Override
public void mouseClicked(MouseEvent e) {
    // [KUSHTI 1] Kontrolllo nëse loja është aktive dhe gjuha nuk
    është në lëvizje
    if (!gameActive || tongueExtending || tongueRetracting)
        return;

    distance clickX = (distX.distT, TONGUY + TONGUE);
    // [KUSHTI 2] Kontrolllo nëse klikimi është brenda rrezes së
    veprimit
    if (dist <= TONGUE_REACH) {
        tongueTargetX = clickX;
        tongueTargetY = clickY;
        tongueExtending = true;
        shootTongue(); // Fillo animacionin e gjuhës
    }
}
```

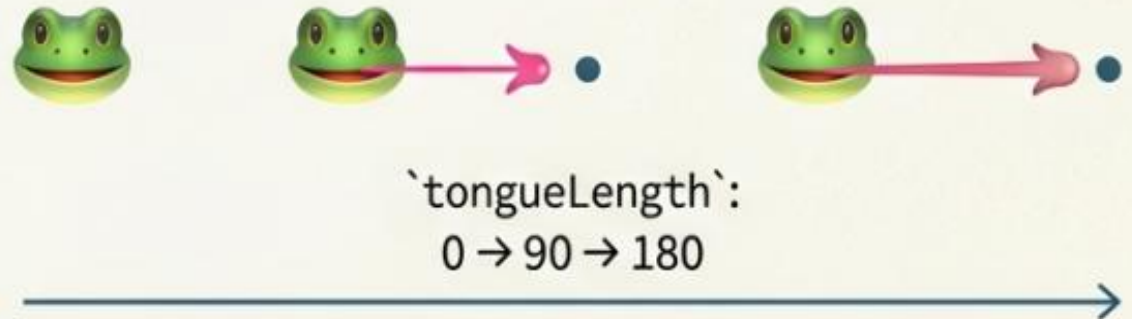


Loja përdor një `MouseListener` për të kapur klikimet. Metoda `mouseClicked` shërben si një portë, duke kontrolluar gjendjen e lojës dhe distancën e klikimit përpara se të aktivizojë veprimin kryesor.

Mekanika Kryesore: Gjuha e Bretkosës (Pjesa 1)

Animacioni i gjuhës nuk është i menjëhershëm. Ai kontrollohet nga `Timer`-i i tij privat dhe dy variabla gjendjeje: `tongueExtending` dhe `tongueRetracting`. Kjo lejon një lëvizje të butë dhe me faza.

```
private void shootTongue() {  
    // ...  
    tongueTimer = new Timer(16, e -> {  
        // Faza 1: Zgjatja  
        if (tongueExtending && tongueLength < targetDist) {  
            tongueLength += 15; // Rrit gjatësinë me 15 pikselë në çdo cikël  
        } else if (tongueExtending) {  
            // Kalimi në fazën tjetër  
            tongueTimer.stop();  
            checkHit();  
            retractTongue();  
        }  
        // ... logjika e tërheqjes ...  
    });  
    tongueTimer.start();  
}
```



Mekanika Kryesore: Goditja dhe Tërheqja (Pjesa 2)

Kontrolli i Goditjes

```
private void checkHit() {  
    // ...  
    for (Bug bug : bugs) {  
        if (bug.isHit(tongueEndX, tongueEndY)) {  
            // Goditje e suksesshme!  
            hitBug = bug;  
            break;  
        }  
    }  
    // ...  
}  
// Brenda klasës Bug  
boolean isHit(int tx, int ty) {  
    double dx = x - tx;  
    double dy = y - ty;  
    // Kontroll distance Euklidiane  
    return Math.sqrt(dx * dx + dy * dy) < 35;  
}
```

`checkHit` iteron mbi të gjitha insektet dhe përdor një kontroll të thjeshtë distance për të parë nëse maja e gjuhës 'godet' ndonjë prej tyre.

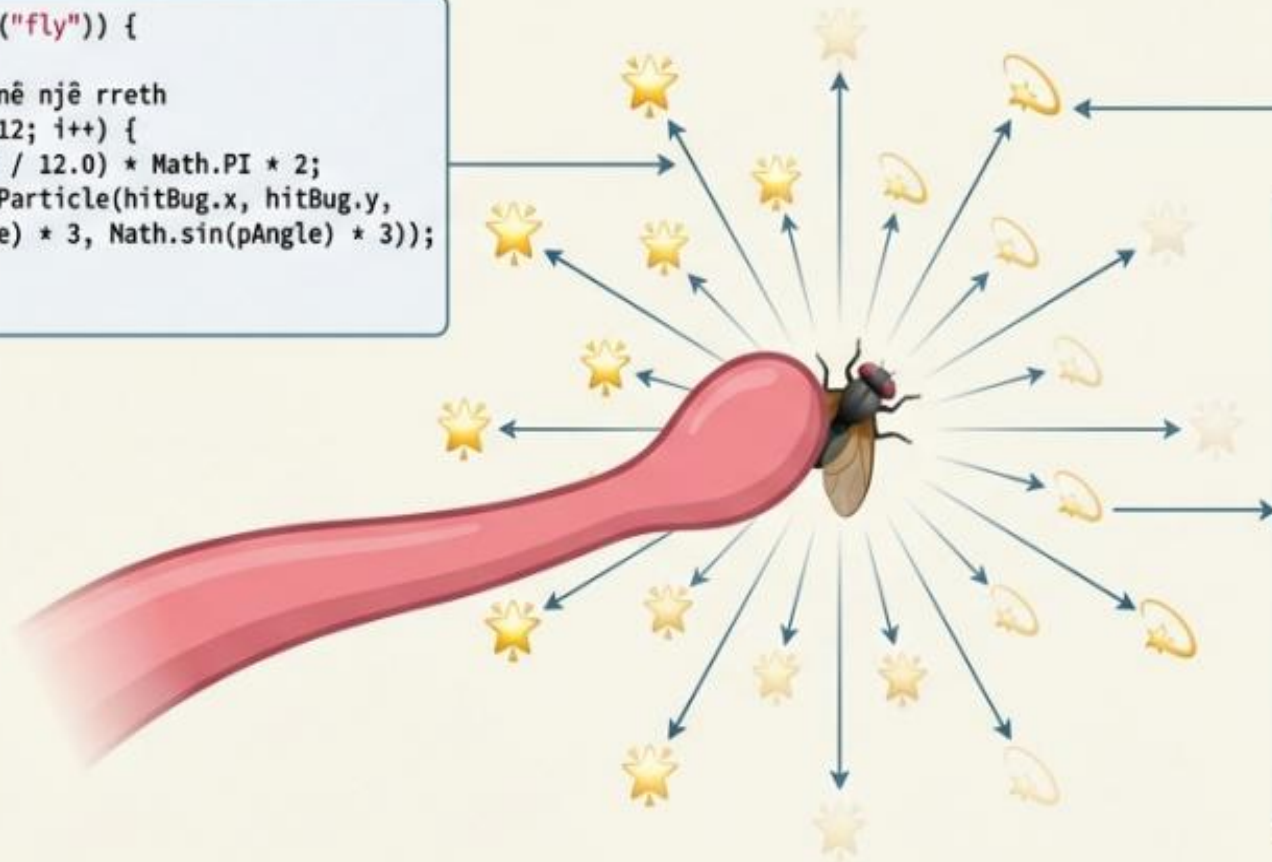
Tërheqja e Gjuhës

```
private void retractTongue() {  
    tongueExtending = false;  
    tongueRetracting = true;  
}  
  
// Brenda Timer-it të gjuhës  
// ...  
else if (tongueRetracting && tongueLength > 0) {  
    tongueLength -= 15; // Zvogëlo gjatësinë  
} else if (tongueRetracting) {  
    tongueLength = 0;  
    tongueRetracting = false; // Kthehu në gjendje pasive  
    tongueTimer.stop();  
}
```

Pas kontrollit, gjendja ndryshon në `tongueRetracting = true` dhe i njëjti `Timer` përdoret për të animuar tërheqjen e gjuhës duke zvogëluar `tongueLength`.

Shpërthimi i Yjeve: Krijimi i Efekteve Vizuale

```
if (hitBug.type.equals("fly")) {  
    score++;  
    // Krijon 12 grimca në një rreth  
    for (int i = 0; i < 12; i++) {  
        double pAngle = (i / 12.0) * Math.PI * 2;  
        particles.add(new Particle(hitBug.x, hitBug.y,  
            Math.cos(pAngle) * 3, Math.sin(pAngle) * 3));  
    }  
}
```



```
class Particle {  
    // ... x, y, vx, vy ...  
    int life = 30; // Jetëgjatësia në cikle  
  
    void update() {  
        x += vx; y += vy; vy += 0.3; // Efekt graviteti  
        life--;  
    }  
  
    void draw(Graphics2D g) {  
        // Zbehet me kalimin e kohës  
        g.setComposite(AlphaComposite.getInstance(AlphaComposite.SRC_OVER, life / 30f));  
        g.drawString(random.nextBoolean() ? "🌟" : "💫",  
            // ...  
        )  
    }  
}
```

Kur një mizë kapet, krijohen disa objekte `Particle`. Secila ka një jetëgjatësi të kufizuar (`life`) dhe një fizikë të thjeshtë. Metoda `draw` përdor `AlphaComposite` për të krijuar një efekt zbehjeje ndërsa grimca "vdes".



Score: 1/3



Misses: 2/5



Time: 74s

Rregullat dhe Ndërfaqja e Përdoruesit (UI)

Variablat e Gjendjes

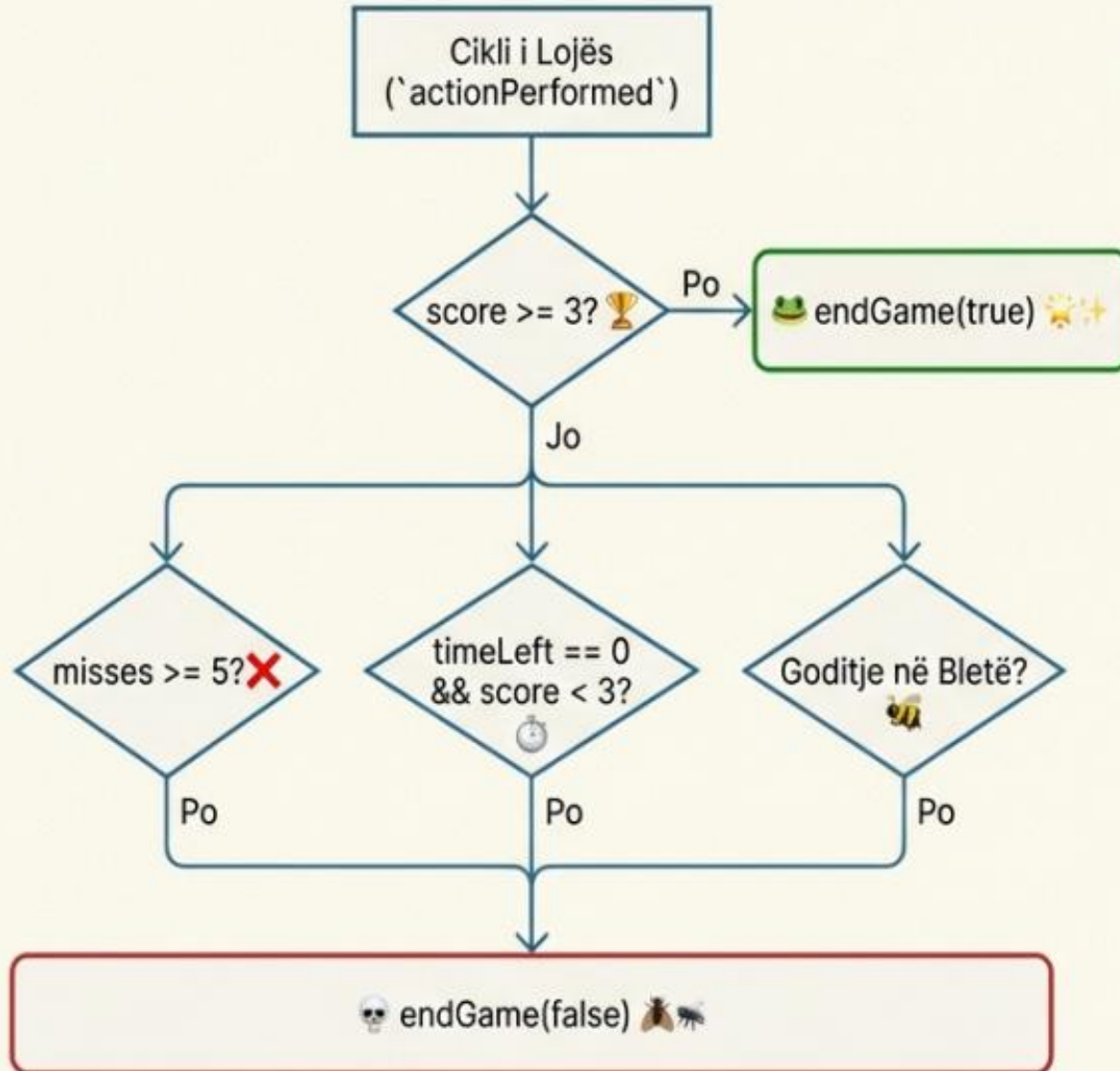
```
private int score = 0;  
private int misses = 0;  
private int timeLeft = 90;  
private boolean gameActive = false;
```

// Vizato UI

```
g2d.setFont(new Font("Arial", Font.BOLD, 18));  
g2d.setColor(Color.WHITE);  
g2d.drawString("🎯 Score: " + score + "/3", 20, 30);  
g2d.drawString("❌ Misses: " + misses + "/5", 250, 30);  
g2d.drawString("🕒 Time: " + timeLeft + "s", 500, 30);
```

Informacioni kritik për lojtarin ruhet në variabla të thjeshta. Në çdo rindërtim të ekranit, këto vlera vizatohen direkt në kanavacë duke përdorur `g2d.drawString`. Kjo siguron që UI-ja është gjithmonë e sinkronizuar me gjendjen aktuale të lojës.

Fitorja dhe Humbja: Kushtet e Përfundimit



Snippet Kodi

```
// Në checkHit, pas një goditjeje të suksesshme në mizë
if (score >= 3) {
    endGame(true); 🏆
}

// Në checkHit, pas një goditjeje në bletë
} else { // hitBug.type is "bee"
    endGame(false); 🐛
}

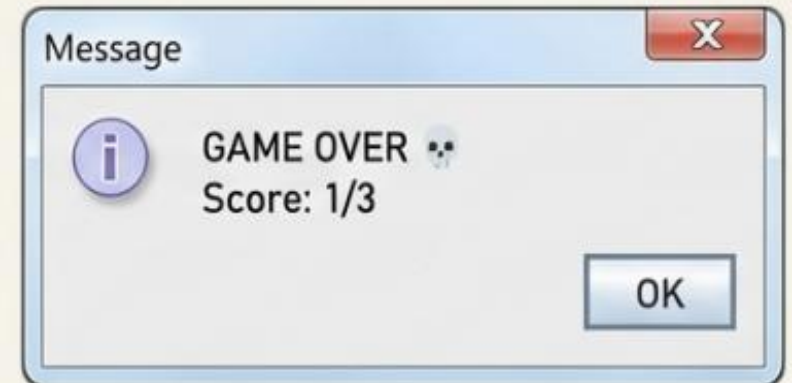
// Në checkHit, pas një gjuajtjeje të pasaktë
} else { // hitBug is null
    misses++;
    if (misses >= 5) {
        endGame(false); ❌
    }
}

// Në actionPerformed
if (timeLeft == 0 && score < 3) {
    endGame(false); ⌚
}
```

Ky kod ilustron saktësisht si funksionon flowchart-i, duke përdorur `endGame(boolean won)` për të përfunduar lojën me rezultatin e duhur bazuar në kushtet e specifikuara.

Mbyllja e Ciklit: `endGame` dhe Rinisja

```
private void endGame(boolean won) {  
    // 1. Ndalo ciklin kryesor të lojës  
    gameActive = false;  
  
    // 2. Krijë mesazhin e përshtatshëm  
    String message = won ? "YOU WIN! 🏆\Score: " + score :  
        "GAME OVER 💀\nScore: " + score + "/3";  
  
    // 3. Shfaq rezultatin të përdoruesit  
    JOptionPane.showMessageDialog(this, message);  
  
    // 4. Rinis lojën nga fillimi  
    startGame();  
}
```



Metoda `endGame` orkestron fundin e lojës në katër hapa të thjeshtë: ndalon përditësimet, formaton një mesazh rezultati, e shfaq atë në një dritare dialogu dhe më pas thërret `startGame()` për të rivendosur të gjitha variablat dhe për të filluar një lojë të re. Kjo krijon një cikël të pafundëm loje.

Konceptet Themelore të Përdorura

Menaxhimi i Gjendjes (State Management)

Përdorimi i variablave boolean (`gameActive`, `tongueExtending`) për të kontrolluar në mënyrë rigoroze fluksin logjik të lojës dhe për të parandaluar veprime të padëshiruara.

```
boolean gameActive = true; boolean tongueExtending = false; 🐸
```

Programimi i Drejtuar nga Ngjarjet (Event-Driven Programming)

Struktura e lojës bazohet në reagimin ndaj ngjarjeve: ngjarje nga `Timer`-at që nxisin përditësimet dhe ngjarje nga `MouseListener` që nxisin veprimet e lojtarit.

Cikli i Lojës (Game Loop) ⌚

Një `Timer` qendror shërben si zemra e lojës, duke siguruar përditësime të vazhdueshme të gjendjes dhe rindërtim të pamjes, gjë që krijon iluzionin e lëvizjes dhe interaktivitetit.

Arkitekturë e Bazuar në Entitete

Ndarja e logjikës në klasa të veçanta (`Bug`, `Particle`) lejon që çdo objekt në lojë të menaxhojë vetë sjelljen e tij (`update`) dhe pamjen (`draw`), duke e bërë kodin më të organizuar dhe më të lehtë për t'u zgjeruar.

Gjenerim Procedural i Përmbajtjes 🌀

Në vend të niveleve të paracaktuara, loja përdor `Timer`-a dhe rastësinë (`Random`) për të gjeneruar vazhdimisht armiq, duke ofruar një sfidë të re sa herë që luhet.



Fatlinda Syla , Bleonita Deskaj & Andeta Leku